



**TRƯỜNG ĐẠI HỌC
CÔNG NGHỆ ĐÔNG Á**
EAST ASIA UNIVERSITY OF TECHNOLOGY

KHOA CÔNG NGHỆ THÔNG TIN

KIỂM THỬ PHẦN MỀM

**Chương 2: Kiểm thử trong vòng đời
phát triển phần mềm**



1. **Hiểu vai trò của kiểm thử** trong từng giai đoạn của vòng đời phát triển phần mềm (Software Development Life Cycle - SDLC).
2. **Nhận biết được các mức độ kiểm thử phần mềm**, bao gồm: kiểm thử đơn vị, kiểm thử tích hợp, kiểm thử hệ thống và kiểm thử chấp nhận.
3. **Phân biệt được các loại kiểm thử** như: kiểm thử chức năng và phi chức năng.
4. **Hiểu được thời điểm và cách thức thực hiện kiểm thử** để tối ưu hiệu quả và chi phí phát triển phần mềm.
5. **Áp dụng kiến thức để xác định chiến lược kiểm thử phù hợp** với từng giai đoạn trong quy trình phát triển.

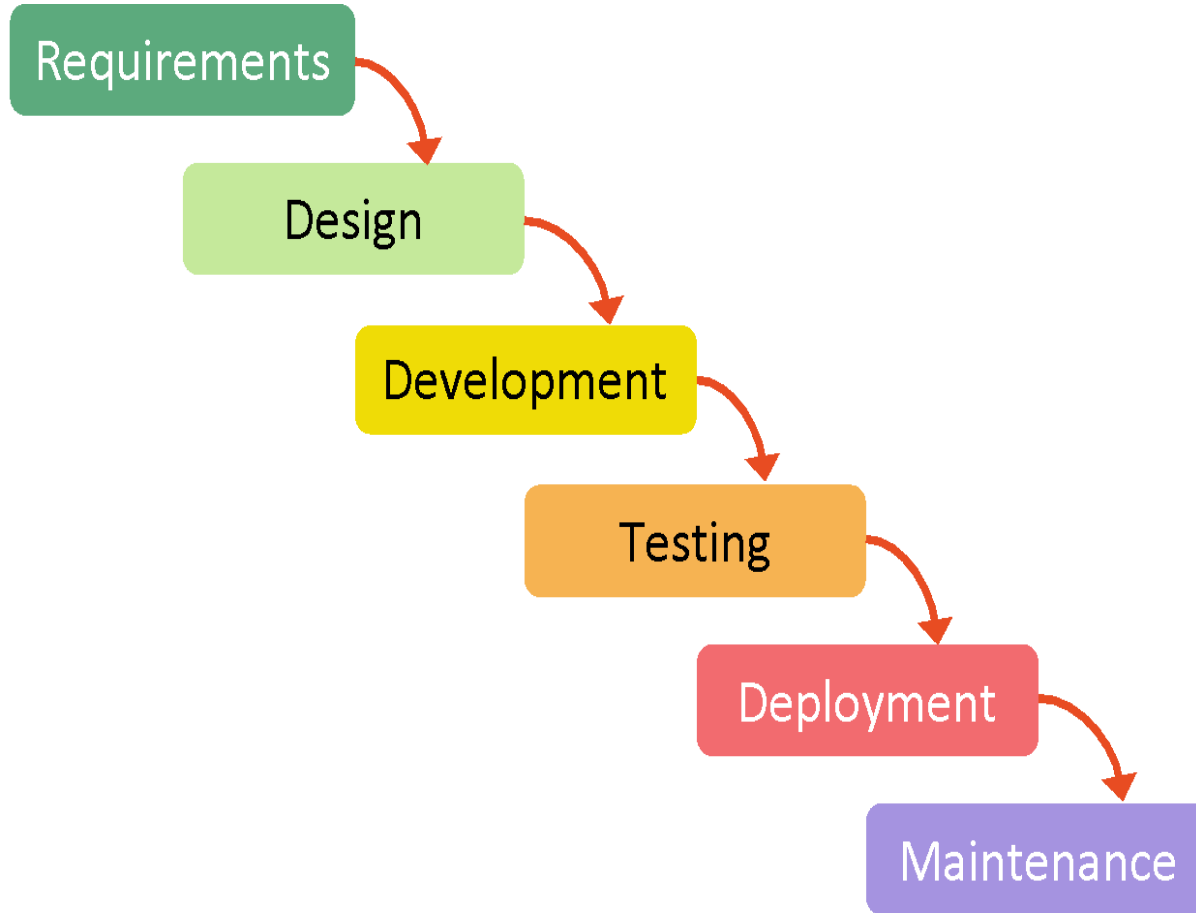
2.1 Kiểm thử trong Ngữ cảnh của Chu kỳ Phát triển Phần mềm

2.2 Các mức độ kiểm thử & Các loại kiểm thử

2.3 Kiểm thử hồi quy

2.1.1 Mô hình thác nước

- Các hoạt động phát triển phần mềm được thực hiện tuần tự.
- Kiểm thử thường diễn ra gần cuối chu kỳ
-> các lỗi được phát hiện gần ngày triển khai thực tế.
- Việc nhận phản hồi và thực hiện sửa đổi khó khăn và có chi phí thay đổi cao.



1. Kiểm thử diễn ra ở cuối chuỗi

Khi dev đã code xong hết rồi tester mới được vào test.

Do đó:

- Nếu có lỗi → sửa lại sẽ rất tốn thời gian và chi phí
- Một lỗi nhỏ phát hiện trễ có thể khiến delay cả hệ thống

Ví dụ: nếu phát hiện yêu cầu sai ở giai đoạn testing → phải quay lại sửa cả phần phân tích, thiết kế, code → vòng lặp chậm.

2. Không nhận được phản hồi sớm

- Vì phải đợi "nước chảy xuống", tester không được tham gia sớm vào quá trình yêu cầu hoặc thiết kế.
- Điều này khiến:
 - Không hiểu rõ nghiệp vụ → test case không đủ
 - Không chủ động thiết kế dữ liệu/môi trường kiểm thử
- Mất cơ hội bắt lỗi sớm ở tài liệu

3. Chi phí sửa lỗi rất cao

Theo nghiên cứu thực tế:

- Sửa lỗi ở giai đoạn yêu cầu tốn 1\$
- Sửa lỗi ở giai đoạn coding tốn 10\$
- Nhưng phát hiện lỗi đó ở giai đoạn testing → tốn 100\$
- Còn nếu để ra production → có thể gây mất khách hàng, mất dữ liệu, uy tín → vô giá

Vậy mô hình thác nước có phù hợp không?



Ưu điểm	Hạn chế
Dễ quản lý, có trình tự rõ ràng	Cứng nhắc, ít phản hồi sớm
Phù hợp với dự án nhỏ, yêu cầu rõ	Không linh hoạt với thay đổi hoặc nghiệp vụ phức tạp

Vì vậy, ngày nay mô hình này ít được dùng trong các dự án hiện đại, nhưng vẫn rất quan trọng để hiểu nguyên lý tổ chức kiểm thử theo trình tự.

Chú ý khi làm đồ án



- Khi học hoặc làm đồ án, nếu áp dụng mô hình thác nước:
 - Hãy thiết kế test case song song khi dev đang code
 - Tham gia đọc tài liệu yêu cầu ngay từ đầu để hiểu sớm
 - Yêu cầu test review thiết kế hoặc logic khi cần

2.1.2 Mô hình chữ V

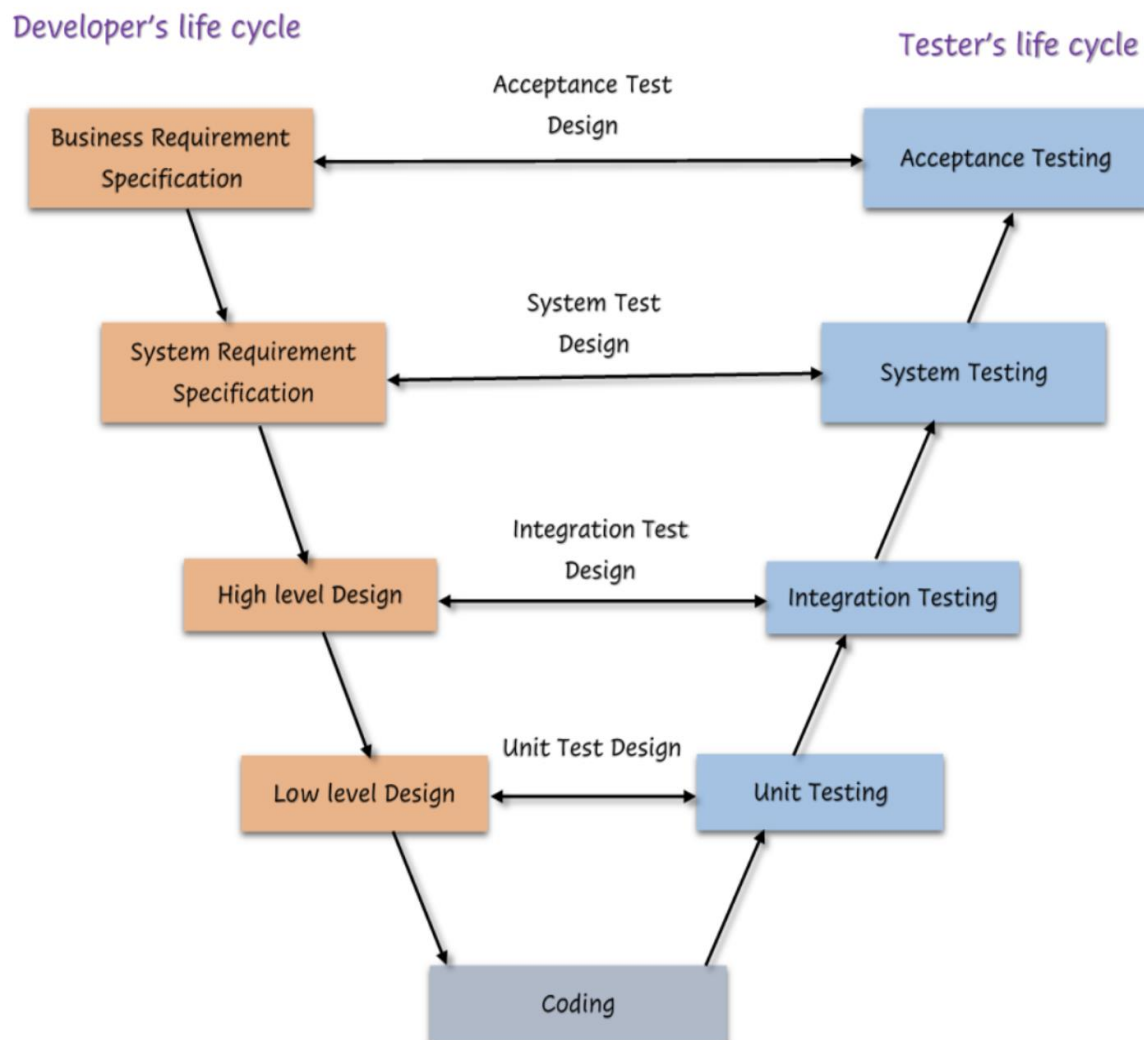


Thiết kế test sớm:

Trong quá trình thiết kế test có thể tìm ra lỗi

-> Tìm ra lỗi sớm thì chi phí fix thấp so với tìm về cuối

-> Phòng ngừa Faults



2.1.2 Mô hình chữ V



Giai đoạn phát triển (Dev)	Kiểm thử tương ứng (Test)	Mục tiêu kiểm tra
☐ Business Requirement	Acceptance Test Design	Kiểm thử mức chấp nhận (từ góc nhìn người dùng)
☐ System Requirement	System Test Design	Kiểm tra toàn bộ hệ thống theo yêu cầu chức năng
☐ High-level Design	Integration Test Design	Kiểm tra kết nối giữa các module/subsystem
☐ Low-level Design	Unit Test Design	Viết test cho từng hàm/chức năng nhỏ
☐ Coding	Chạy thực tế các Unit Test / Build	Thực hiện toàn bộ test theo thiết kế trước đó

1. Thiết kế test sớm

- Ngay khi phân tích yêu cầu đã bắt đầu viết test case, thiết kế kiểm thử
- Có thể phát hiện sớm những mâu thuẫn, thiếu sót hoặc yêu cầu không rõ ràng

Ví dụ:

- Khi BA viết yêu cầu "sinh viên đủ điểm thì được tốt nghiệp", nếu tester tham gia và hỏi:
 - "Đủ là bao nhiêu điểm? GPA, tín chỉ, điểm F có tính không?"
→ Là lỗi nghiệp vụ phát hiện sớm!

2. Chi phí sửa lỗi thấp

- Lỗi tìm ra trong giai đoạn tài liệu → sửa rất rẻ
- Lỗi tìm ra sau khi coding → sửa tốn công, tốn thời gian → dễ ảnh hưởng dây chuyền

3. Phòng ngừa lỗi (fault prevention)

- Test được thiết kế từ đầu nên hệ thống có khả năng chống lỗi tốt hơn
- Đảm bảo chất lượng không chỉ bằng việc "test sau", mà kiểm soát từ đầu chu trình

Ưu điểm nổi bật của mô hình chữ V



- Mô hình chữ V là một bước tiến tư duy kiểm thử sớm – thiết kế kiểm thử chuyên nghiệp.
- Thay vì phản ứng bị động với lỗi → mô hình này giúp QA trở thành người chủ động phòng ngừa và nâng cao chất lượng phần mềm.

Chú ý khi làm đồ án



- Hãy thiết kế test case ngay khi đọc đặc tả
- Đề xuất test cho từng chức năng tương ứng với tài liệu
- Làm việc song song với dev, không chờ tới cuối mới nhảy vào test

2.1.3 Mô hình Agile-Scrum

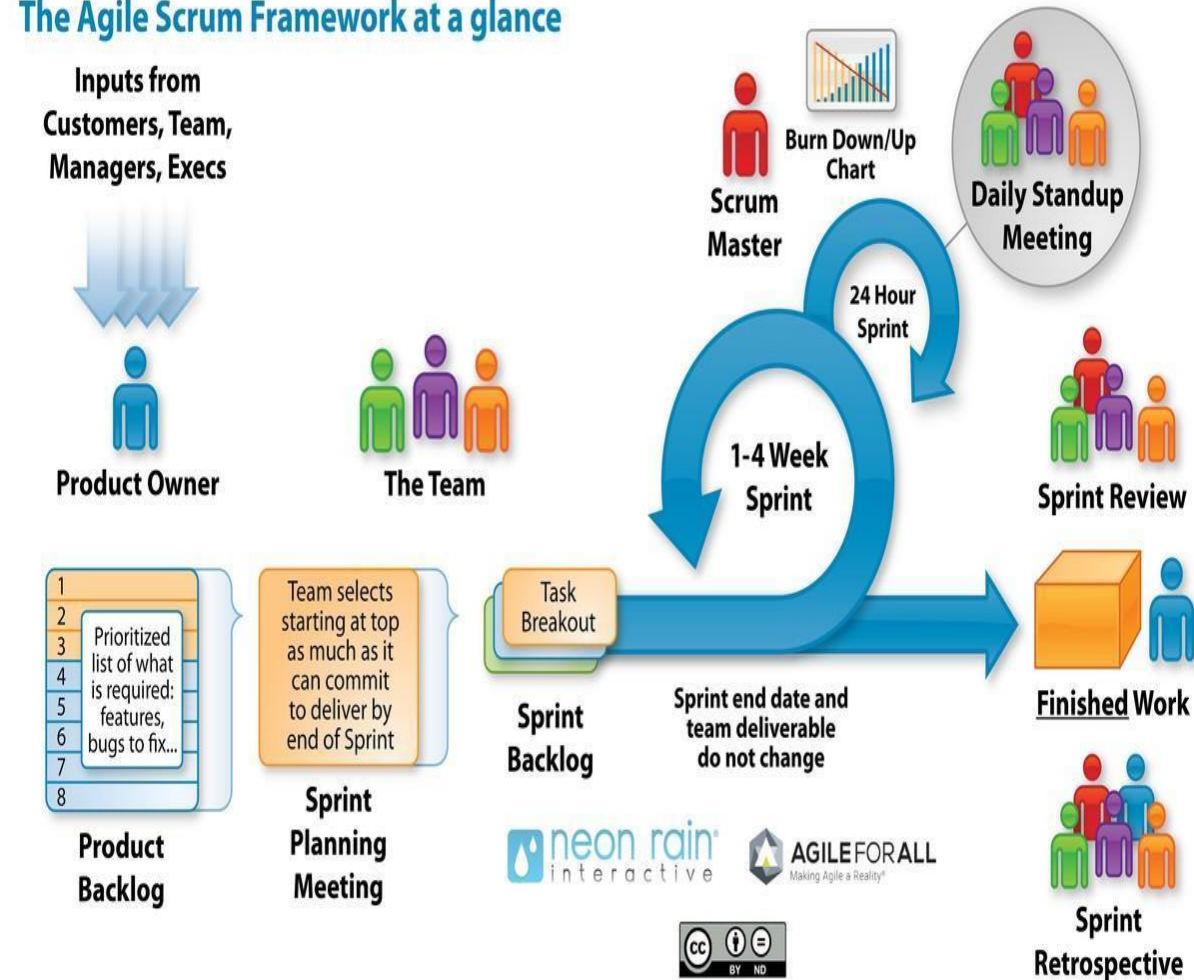
- Agile = Phát triển lặp + Phát triển tăng
- Phát triển lặp (iteration): Làm theo từng vòng ngắn (sprint)
- Phát triển tăng (incremental): Mỗi vòng làm ra một phần có thể dùng được
- Mỗi sprint thường kéo dài 1-4 tuần

Tester không đợi dev code xong, mà cùng làm trong cùng sprint

2.1.3 Mô hình Agile-Scrum

- Mỗi vòng lặp có thời gian ngắn
- Sự tăng trưởng của các tính năng tương ứng nhỏ (cải tiến tính năng cũ và/hoặc thêm 2 - 3 tính năng mới).
- Không quá coi trọng việc tạo tài liệu cho các sản phẩm công việc.
- Sử dụng tự động hóa kiểm thử một cách rộng rãi để làm cho việc kiểm thử hồi quy trở nên dễ dàng. Hầu hết kiểm thử thủ công thường được thực hiện bằng các kỹ thuật kiểm thử dựa trên kinh nghiệm.

The Agile Scrum Framework at a glance



2.1.3 Mô hình Agile-Scrum



Giai đoạn	Nội dung
☐ Product Backlog	Danh sách tất cả các tính năng/phản hồi cần làm (do PO quản lý)
☐ Sprint Planning	Nhóm chọn ra một phần backlog để làm trong Sprint tiếp theo
☐ Sprint (1–4 tuần)	Làm việc, coding, testing liên tục. Tester không đợi dev xong mới test, mà làm song song
☐ Daily Standup	Mỗi ngày họp 15 phút: Hôm qua làm gì, hôm nay làm gì, có vướng mắc gì không? Tester phải báo cáo bug hoặc tiến độ test
☐ Sprint Review	Demo tính năng đã hoàn thành để lấy phản hồi từ PO & khách hàng
☐ Sprint Retrospective	Nhìn lại cả sprint để rút kinh nghiệm: cái gì tốt, cái gì cần cải thiện?

Vai trò của Product Owner (PO)



Nhiệm vụ chính	Mô tả
☐ Quản lý Product Backlog	Sắp xếp và ưu tiên các tính năng cần phát triển
☐ Là cầu nối giữa khách hàng và team phát triển	Giải thích nghiệp vụ, làm rõ yêu cầu
☐ Đưa ra quyết định về phạm vi, deadline và giá trị sản phẩm	Có quyền quyết định cái gì được làm trước, cái gì có thể bỏ qua
☐ Tham gia Sprint Planning, Review, Retro	Đảm bảo tính năng mang lại giá trị thực tế và đúng nhu cầu người dùng

1. Kiểm thử là một phần không thể tách rời của team

- Tester làm việc mỗi ngày cùng dev
- Tham gia từ Sprint Planning → biết phải test gì, chuẩn bị test case từ đầu
- Họp hằng ngày để báo cáo tình trạng bug

2. Ưu tiên tự động hóa kiểm thử

- Vì mỗi sprint là một vòng lặp ngắn, nên phải test hồi quy liên tục
- Do đó, việc viết script kiểm thử (UI/API) để tự chạy test là cực kỳ quan trọng
- Tester cần biết dùng: Selenium, Postman, JMeter, Playwright...

3. Không tốn quá nhiều tài liệu

- Agile không yêu cầu viết test plan dài 20 trang
- Chỉ cần test case đủ dùng và cập nhật thường xuyên
- Kiểm thử dựa nhiều vào kỹ năng, kinh nghiệm và sự hợp tác trong nhóm

Lợi ích	Vì sao?
Phát hiện lỗi sớm	Do tester tham gia ngay từ khi lập kế hoạch
Tăng tốc độ phản hồi	Mỗi sprint test xong, có thể release luôn
Test được cải tiến liên tục	Mỗi sprint là cơ hội để rút kinh nghiệm, cải tiến quy trình test

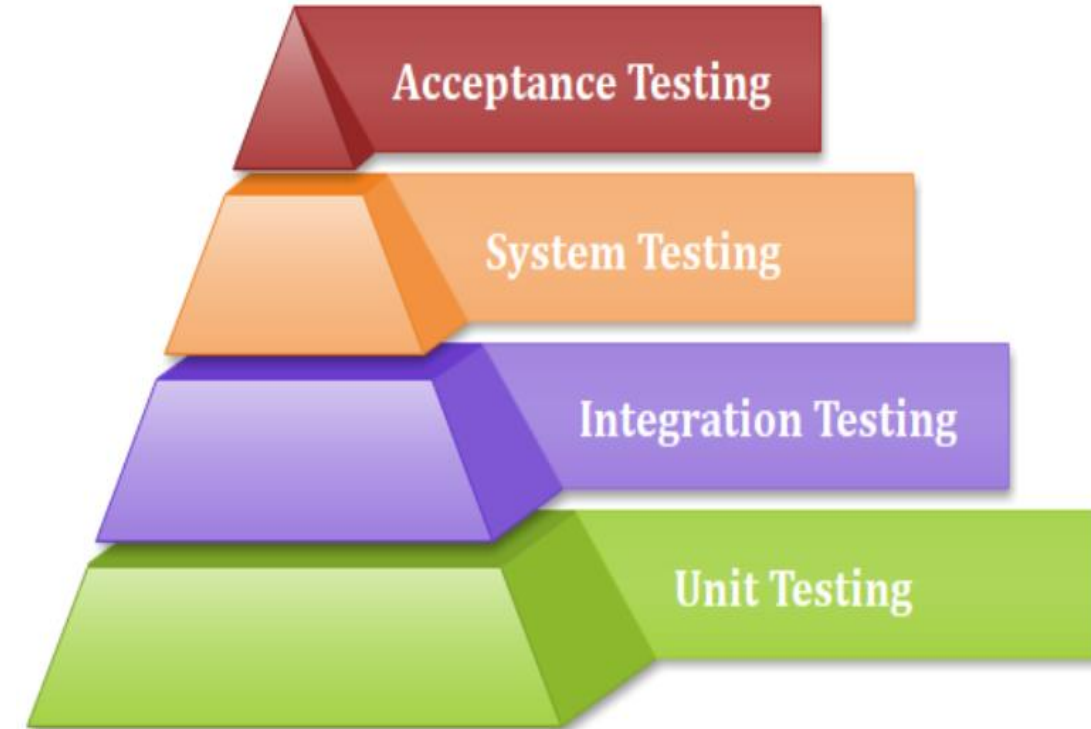
2.1 Kiểm thử trong Ngữ cảnh của Chu kỳ Phát triển Phần mềm

2.2 Các mức độ kiểm thử & Các loại kiểm thử

2.3 Kiểm thử hồi quy

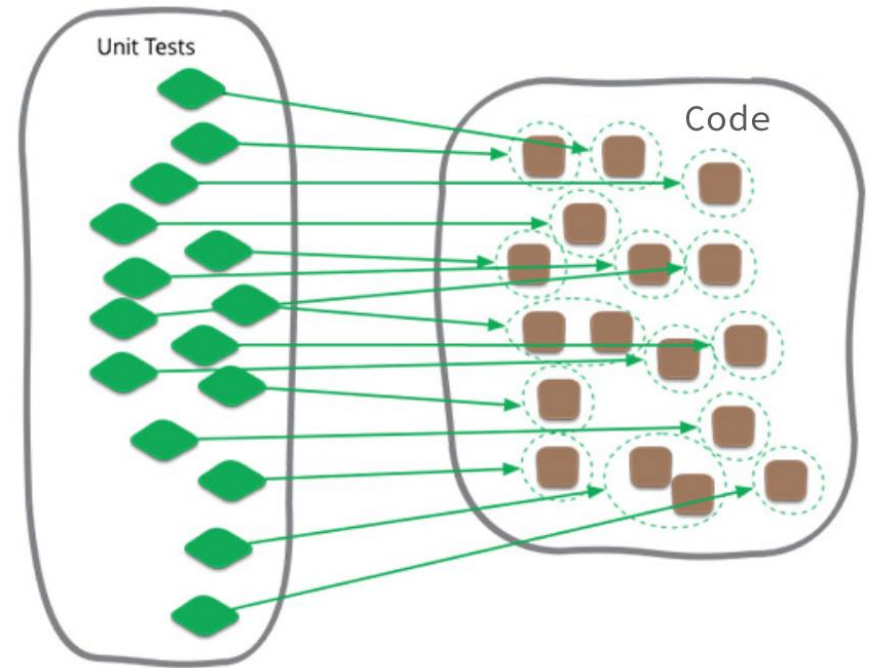
2.2.1 Các mức độ kiểm thử

- Mức độ kiểm thử (Testing Levels) là cách chia các hoạt động kiểm thử thành từng nhóm rõ ràng, mỗi nhóm được thực hiện ở một giai đoạn khác nhau trong vòng đời phát triển phần mềm.
- Mức độ kiểm thử liên quan đến các hoạt động tương ứng trong chu kỳ phát triển phần mềm.
- Các mức độ này phản ánh cách phần mềm được lắp ráp và kiểm tra từ nhỏ đến lớn, từ từng hàm riêng lẻ đến toàn hệ thống hoàn chỉnh.



2.2.1 Các mức độ kiểm thử

- **Kiểm thử đơn vị (Unit/Component testing)**
 - Tập trung vào kiểm thử các thành phần độc lập của phần mềm.
 - Thường yêu cầu hỗ trợ cụ thể như framework kiểm thử đơn vị.
 - Thông thường, kiểm thử thành phần được thực hiện bởi những lập trình viên trên môi trường phát triển của họ.



- Kiểm thử đơn vị là: Là kiểm thử các đơn vị nhỏ nhất của phần mềm – thường là một hàm (function), phương thức (method) hoặc một class/module riêng biệt.
- Mỗi đơn vị được test tách biệt với phần còn lại của hệ thống.
- Mục tiêu là: phát hiện lỗi logic nhỏ, lỗi xử lý biên, lỗi cú pháp... càng sớm càng tốt.

Đặc điểm Unit testing



- Tập trung vào kiểm thử các thành phần độc lập của phần mềm.
- Thường yêu cầu hỗ trợ cụ thể như framework kiểm thử đơn vị, thường được thực hiện bởi lập trình viên trong môi trường phát triển của họ lập trình viên thường dùng các công cụ sau:

Ngôn ngữ	Framework Unit Test phổ biến
Java	JUnit
Python	Pytest, Unittest
C#	NUnit
JavaScript	Jest, Mocha

Lợi ích của Unit Testing



Lợi ích	Vì sao?
Phát hiện lỗi sớm	Do test chạy ngay sau khi viết mã
Tăng tốc phát triển	Giúp refactor code mà không lo phá vỡ chức năng
Hỗ trợ kiểm thử hồi quy	Có thể tích hợp vào pipeline CI để test mỗi lần commit
Cơ sở cho kiểm thử tích hợp	Unit test tốt → nền tảng vững chắc cho Integration test

Objectives	Giảm thiểu rủi ro. Xác minh hành vi chức năng và phi chức năng. Tăng độ tin cậy. Phát hiện lỗi. Ngăn ngừa lỗi.
Test Basis	Thiết kế chi tiết. Mã nguồn. Mô hình dữ liệu. Tài liệu đặc tả thành phần.
Test Objects	Thành phần, đơn vị, module. Cấu trúc mã và dữ liệu. Lớp. Mô hình cơ sở dữ liệu.
Typical Defects & Failures	Chức năng sai. Lỗi luồng dữ liệu. Lỗi logic hoặc mã sai.
Approaches & Responsibilities	Thường được thực hiện bởi lập trình viên

Mối liên hệ kiểm thử đơn vị với các mức độ khác

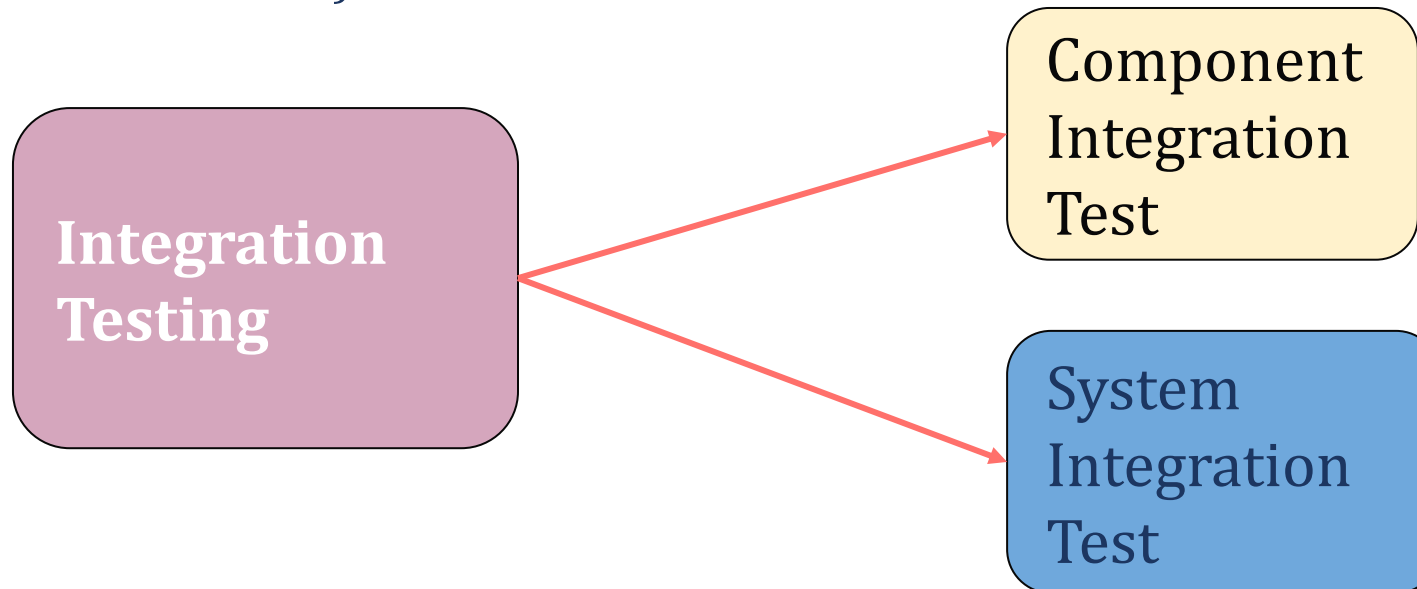


- Unit test là đáy của kim tự tháp kiểm thử
- Nếu không có unit test tốt → lỗi sẽ “lọt” lên tầng integration/system test → tốn kém sửa lỗi hơn
- -> Một tester giỏi không chỉ biết test hệ thống – mà còn hiểu và biết cách hỗ trợ (hoặc viết) unit test, nhất là khi làm việc trong môi trường Agile, CI/CD hoặc DevOps.

2.2.1 Các mức độ kiểm thử

Kiểm thử tích hợp (Integration Testing)

- Integration Testing được chia thành 2 nhánh chính:
 - Component Integration Testing – kiểm thử tích hợp giữa các module nội bộ trong hệ thống
 - System Integration Testing – kiểm thử tích hợp giữa hệ thống phần mềm với hệ thống bên ngoài (API, payment, service,...)



2.2.1 Các mức độ kiểm thử

Kiểm thử tích hợp - Component integration testing

- Tập trung vào kiểm thử interfaces and interactions giữa các component, tập trung vào các điểm tiếp xúc giữa các thành phần (components) trong cùng một hệ thống phần mềm.
- Chiến lược tích hợp: bottom-up, top-down hay big-bang

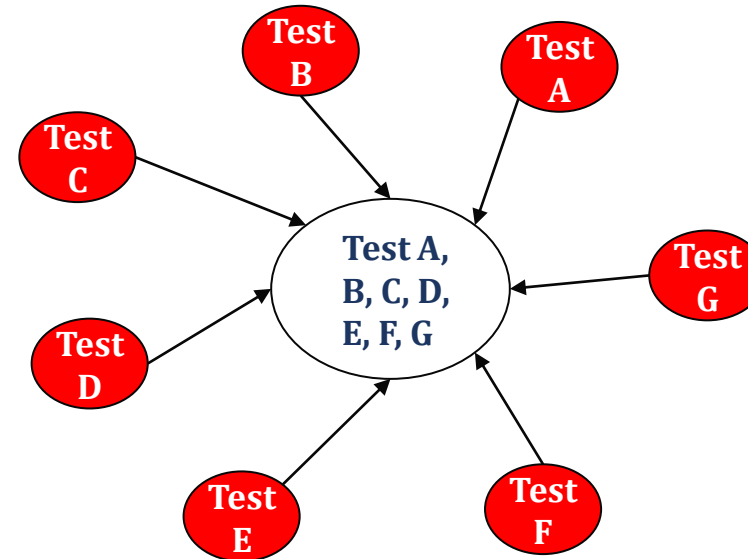
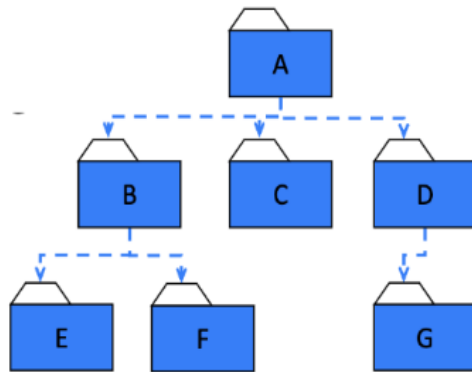
Ví dụ hệ thống xét tốt nghiệp



- Dev viết riêng:
 - Module tính GPA
 - Module kiểm tra học phần không có điểm F
 - Module kiểm tra QPAN, GDTC
 - Module tổng hợp điều kiện tốt nghiệp
- → Từng cái test bằng unit test
- → Khi tích hợp các module đó lại → phải dùng Component Integration Testing để chắc chắn toàn bộ chain xử lý không lỗi.

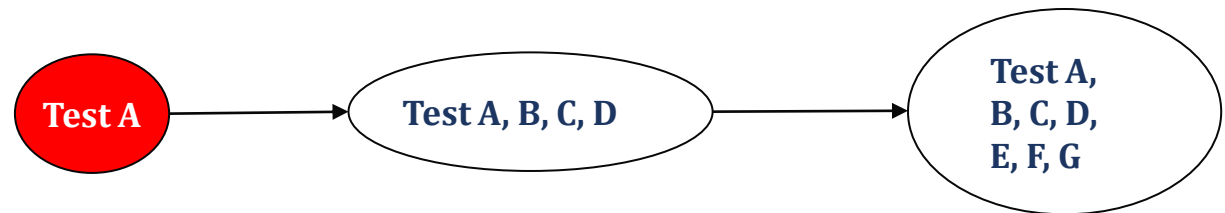
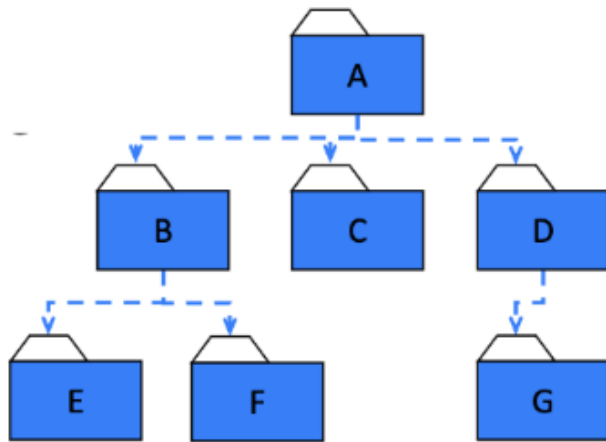
2.2.1 Các mức độ kiểm thử

- Kiểm thử tích hợp - Component integration testing - Chiến lược tích hợp Big bang: Ghép tất cả các module lại cùng lúc, rồi test toàn bộ một lượt. -> Nguy hiểm vì khó xác định lỗi thuộc module nào, chỉ dùng khi hệ thống nhỏ, hoặc gấp rút



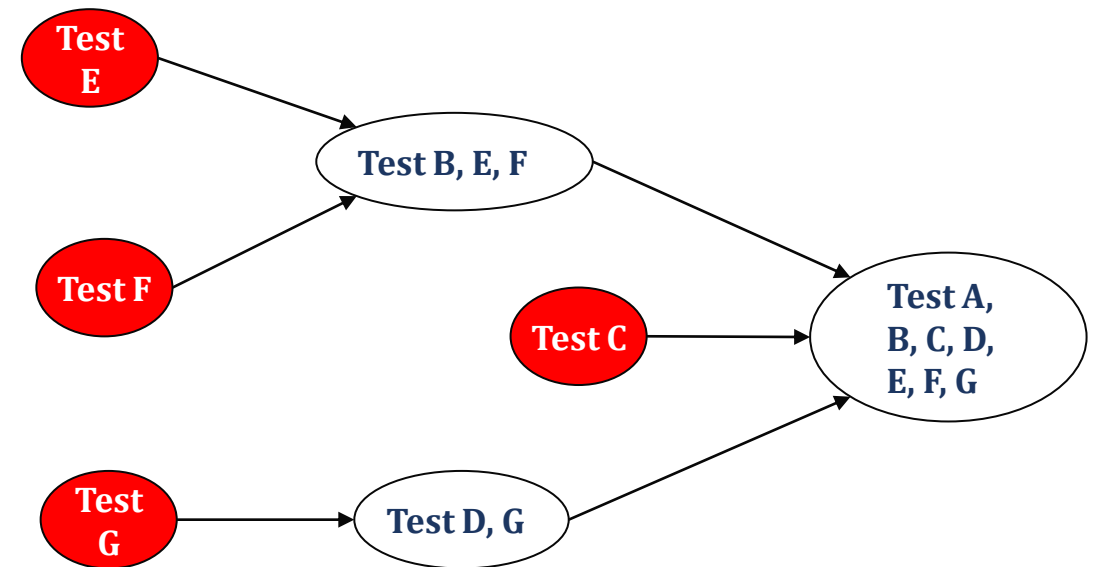
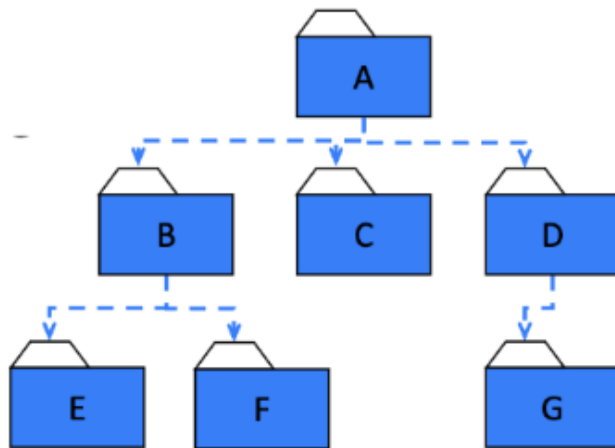
2.2.1 Các mức độ kiểm thử

- Kiểm thử tích hợp - Component integration testing - Chiến lược tích hợp Top-down:
Test từ các module cao cấp nhất (UI, controller) rồi tích hợp dần các module con bên dưới. Phù hợp khi muốn kiểm tra sớm hành vi tổng thể



2.2.1 Các mức độ kiểm thử

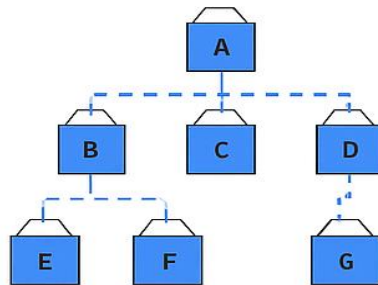
- Kiểm thử tích hợp - Component integration testing - Chiến lược tích hợp Bottom-up, Test từ các module thấp nhất trước (gần CSDL, xử lý logic), sau đó dần ghép lên các module điều phối. Để kiểm soát lỗi từng phần nhỏ.



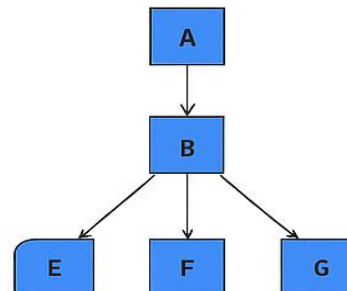
Ví dụ: Hệ thống quản lý điểm sinh viên

Module	Chức năng
A	A – Giao diện người dùng
B	B – Quản lý sinh viên
C	C – Quản lý môn học
D	D – Tính điểm trung bình
E	E – Xếp loại học lực
F	F – Xếp loại học lực
G	G – Xuất bảng điểm PDF

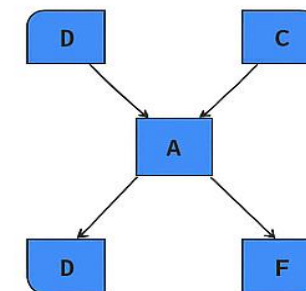
Big bang
Tích hợp toàn bộ
các module cùng lúc



Top-down
Tích hợp từ module
cấp cao xuống



Bottom-up
Tích hợp từ module
cấp thấp lên



2.2.1 Các mức độ kiểm thử

Kiểm thử tích hợp - System integration testing

- Tập trung vào kiểm thử interfaces của system với các system khác và các dịch vụ bên ngoài.
- Đây là kiểm thử sự tích hợp giữa toàn bộ hệ thống của bạn với:
 - Một hoặc nhiều hệ thống bên ngoài (third-party systems)
 - Các dịch vụ web/API (REST, SOAP, GraphQL...)
 - Các nền tảng khác (cổng thanh toán, thư viện, bên thứ 3)
- Không còn test từng module hay từng hàm → mà là test toàn bộ hệ thống giao tiếp với bên ngoài.

- Trong hệ thống “Quản lý điểm sinh viên”, khi xét tốt nghiệp, có thể cần:
 - Gửi dữ liệu kết quả lên cổng Bộ GD-ĐT → qua API
 - Xác thực thông tin SV từ hệ thống quản lý sinh viên tập trung
 - In phiếu xét tốt nghiệp thông qua tích hợp với hệ thống in ấn nội bộ
- → Tất cả đều cần System Integration Testing

Mục tiêu của System Integration Testing



Mục tiêu cụ thể	Ví dụ kiểm thử
Kiểm tra giao tiếp API	Gửi request từ hệ thống A → nhận đúng response từ hệ thống B
Kiểm tra xác thực, token	Kiểm tra hệ thống có sử dụng đúng token/API key
Kiểm tra timeout, retry	Nếu dịch vụ B chậm, hệ thống A có xử lý retry/lỗi hợp lý không
Kiểm tra mapping dữ liệu	Dữ liệu đầu ra đúng format, đúng cấu trúc

Các lỗi thường gặp



Lỗi	Mô tả
Sai endpoint	Gọi nhầm địa chỉ dịch vụ
Mapping sai	Dữ liệu gửi khác schema yêu cầu
Timeout	Dịch vụ ngoài chậm hoặc không phản hồi
Sai token/ authorization	Không xác thực được

- Đây là một trong những bước kiểm thử có độ rủi ro cao nhất, vì liên quan đến yếu tố mà bạn không thể kiểm soát hoàn toàn – ví dụ hệ thống bên ngoài thay đổi mà bạn không biết.
- Do đó:
 - Cần có môi trường staging/sandbox mô phỏng thật
 - Cần log kỹ giao tiếp (request/response)
 - Cần cả manual test và automation test

2.2.1 Các mức độ kiểm thử



Kiểm thử hệ thống (System testing)

- Tập trung vào hành vi tổng thể (overall behavior) và khả năng (capabilities) của một hệ thống hoặc sản phẩm.
- Thường bao gồm kiểm thử chức năng của nghiệp vụ từ đầu đến cuối (end-to-end test) và kiểm thử phi chức năng (non-functional) của các đặc tính chất lượng (quality characteristics).
- Môi trường test phải ổn định, phù hợp, tốt nhất là tương tự như môi trường vận hành.

- Điểm khác biệt rõ nhất với Unit hay Integration Testing
 - Unit test: kiểm thử từng hàm, từng class nhỏ
 - Integration test: kiểm tra module kết nối với nhau
 - System test: chạy thử toàn hệ thống như người dùng thật sẽ sử dụng
- Mục tiêu của System Testing
 - Hệ thống có chạy đúng không?
 - Tính năng hoạt động có đúng như tài liệu mô tả không?
 - Giao diện, chức năng, dữ liệu có phản hồi đúng không?

- Thường bao gồm kiểm thử chức năng từ đầu đến cuối và kiểm thử phi chức năng
- Kiểm thử chức năng (Functional Test) → Kiểm tra hệ thống có thực hiện đúng theo yêu cầu nghiệp vụ không. Còn gọi là end-to-end test:

Ví dụ trong hệ thống xét tốt nghiệp:

- Login → xem thông tin → xét điều kiện tốt nghiệp → xuất báo cáo → gửi email thông báo → lưu lịch sử
- Toàn bộ luồng nghiệp vụ đó chính là 1 ca kiểm thử end-to-end.

- Kiểm thử phi chức năng (Non-functional Test) → Dù hệ thống chạy đúng, nhưng nếu:
 - Chậm, không ổn định, dễ bị sập, dễ bị hack...
 - Giao diện không thân thiện
 - Không phản hồi được nhiều người dùng cùng lúc
- Vì thế, System Testing còn kiểm thử:
 - Hiệu năng (performance)
 - Bảo mật (security)
 - Khả năng phục hồi (recovery)
 - Khả năng sử dụng (usability)
 - Khả năng tương thích (compatibility)...

2.2.1 Các mức độ kiểm thử



■ Kiểm thử hệ thống (System testing)

Mục	Nội dung
Mục tiêu	Giảm rủi ro. Xác minh hành vi chức năng và phi chức năng của hệ thống. Xác nhận hệ thống hoàn chỉnh và đúng như mong đợi. Xây dựng sự tin tưởng. Tìm và ngăn ngừa lỗi.
Cơ sở kiểm thử	Tài liệu yêu cầu phần mềm và hệ thống. Báo cáo phân tích rủi ro. Các trường hợp sử dụng. Các Epic và user story. Mô hình hệ thống. Biểu đồ trạng thái. Hướng dẫn sử dụng hệ thống và phần mềm.
Đối tượng kiểm thử	Ứng dụng. Phần cứng/phần mềm. Hệ điều hành. Hệ thống đang được kiểm thử (SUT). Cấu hình hệ thống và dữ liệu cấu hình.
Lỗi và sự cố thường gặp	Tính toán sai. Hành vi hệ thống không chính xác hoặc không mong muốn (chức năng/phi chức năng). Luồng dữ liệu sai. Không thể hoàn thành các tác vụ từ đầu đến cuối. Không đúng như mô tả trong hướng dẫn.
Cách tiếp cận và trách nhiệm	Đảm bảo hệ thống hoạt động đúng yêu cầu, bao gồm chức năng & phi chức năng. Dựa vào yêu cầu nghiệp vụ và kỹ thuật để lập kế hoạch kiểm thử. Ghi nhận, theo dõi và phối hợp sửa lỗi kịp thời.

- Kiểm thử hệ thống (System testing) trong bài toán “Quản lý điểm sinh viên”:
 - Test nhập điểm → tính GPA → xét tốt nghiệp → in phiếu
 - Test người dùng sai thao tác có được xử lý hợp lý không
 - Test tốc độ phản hồi khi 100 sinh viên đồng loạt truy cập

2.2.1 Các mức độ kiểm thử

Kiểm thử chấp nhận (Acceptance testing)

- Tập trung vào việc validation và chứng minh sự sẵn sàng triển khai, điều này có nghĩa là hệ thống đáp ứng đúng nhu cầu nghiệp vụ của người dùng.
- Lý tưởng nhất, kiểm thử chấp nhận nên được thực hiện bởi người dùng. Các hình thức chính của kiểm thử chấp nhận bao gồm:
 - Kiểm thử chấp nhận người dùng (UAT)
 - Kiểm thử chấp nhận vận hành (Operational Acceptance Testing)
 - Kiểm thử chấp nhận theo hợp đồng và các quy định (contractual and regulatory acceptance testing)
 - Kiểm thử alpha và beta

2.2.1 Các mức độ kiểm thử

■ Kiểm thử chấp nhận (Acceptance testing)

Objectives	Xây dựng sự tin tưởng. Xác nhận hệ thống đã hoàn chỉnh và hoạt động đúng như mong đợi. Kiểm tra các hành vi chức năng và phi chức năng theo đúng yêu cầu.
Test Basis	- Quy trình nghiệp vụ- Yêu cầu người dùng/doanh nghiệp- Quy định pháp lý, hợp đồng, tiêu chuẩn- Tài liệu hệ thống, tài liệu người dùng- Báo cáo phân tích rủi ro- Quy trình backup & khôi phục- Kế hoạch dự phòng- Yêu cầu phi chức năng- Mục tiêu vận hành- Gói CSDL- Tiêu chuẩn bảo mật
Test Objects	- Hệ thống được kiểm thử (SUT)- Cấu hình và dữ liệu cấu hình- Cơ chế khôi phục dữ liệu- Tình huống khẩn cấp (Hot sites)- Form nhập liệu- Báo cáo
Typical Defects & Failures	- Quy trình hệ thống không đúng - Quy tắc nghiệp vụ sai - Không tuân thủ hợp đồng - Lỗi phi chức năng: bảo mật, hiệu năng, ổn định
Approaches & Responsibilities	Xây dựng sự tin tưởng. Xác minh hệ thống đã hoàn chỉnh và hoạt động như mong đợi. Kiểm tra các hành vi chức năng và phi chức năng đã được mô tả.

2.2.2 Các loại kiểm thử

Kiểm thử chức năng (Functional testing)

- Đánh giá các chức năng mà một thành phần hoặc hệ thống nên thực hiện.
- Những chức năng là những thứ "what" mà đối tượng kiểm thử nên thực hiện.
- Mục tiêu chính của kiểm thử chức năng là kiểm tra độ hoàn thiện chức năng, tính chính xác chức năng và sự thích hợp chức năng.

2.2.2 Các loại kiểm thử

Kiểm thử phi chức năng (Non-functional testing)

- Đánh giá các thuộc tính khác ngoài đặc điểm chức năng của một thành phần (component) hoặc hệ thống (system).
- Kiểm thử phi chức năng là việc kiểm thử "cách hệ thống hoạt động" ("how well the system behaves").
- Mục tiêu chính của kiểm thử phi chức năng là kiểm tra các đặc tính chất lượng phi chức năng của phần mềm (non-functional software quality characteristics).

2.2.2 Các loại kiểm thử



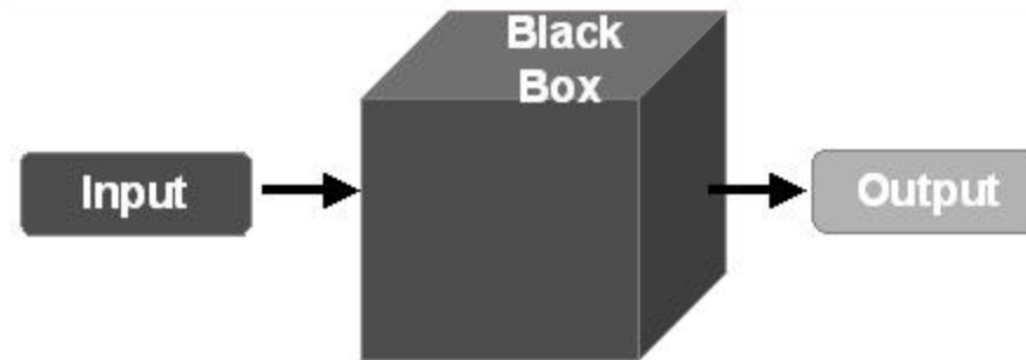
- Đặc tính chất lượng phi chức năng của phần mềm (Non-functional software quality characteristics)
 - Hiệu suất hiệu quả (Performance efficiency)
 - Tương thích (Compatibility)
 - Sử dụng được (Usability)
 - Tin cậy (Reliability)
 - Bảo mật (Security)
 - Dễ bảo trì (Maintainability)
 - Khả năng di động (Portability)

2.2.2 Các loại kiểm thử



Kiểm thử hộp đen (Black-box testing)

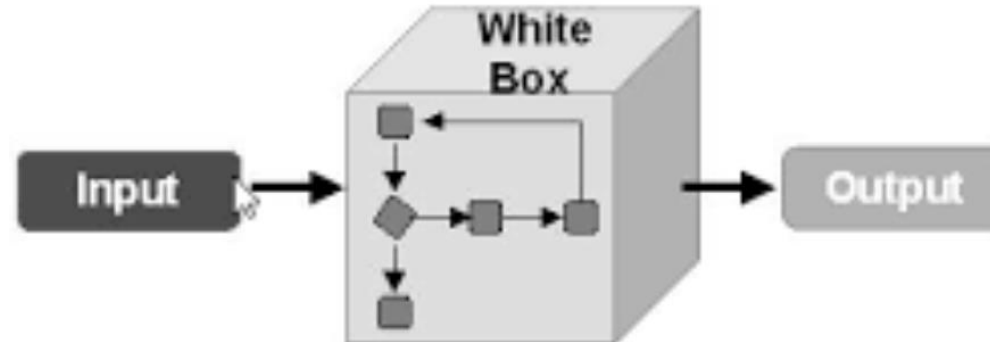
- Dựa trên đặc tả yêu cầu và các test được xác định từ tài liệu bên ngoài đối tượng test.
- Mục tiêu chính của kiểm thử hộp đen là kiểm tra hành vi của hệ thống so với các thông số đặc tả của nó.



2.2.2 Các loại kiểm thử

Kiểm thử hộp trắng (White-box testing)

- Là kiểu kiểm thử dựa trên cấu trúc và các tests được xác định từ mã cài đặt của hệ thống (system's implementation) hoặc cấu trúc nội bộ (ví dụ: code, architecture, work flows, và data flows).
- Mục tiêu chính của kiểm thử hộp trắng là bao phủ cấu trúc cơ bản thông qua các tests ở mức chấp nhận được.



2.2.2 Các loại kiểm thử



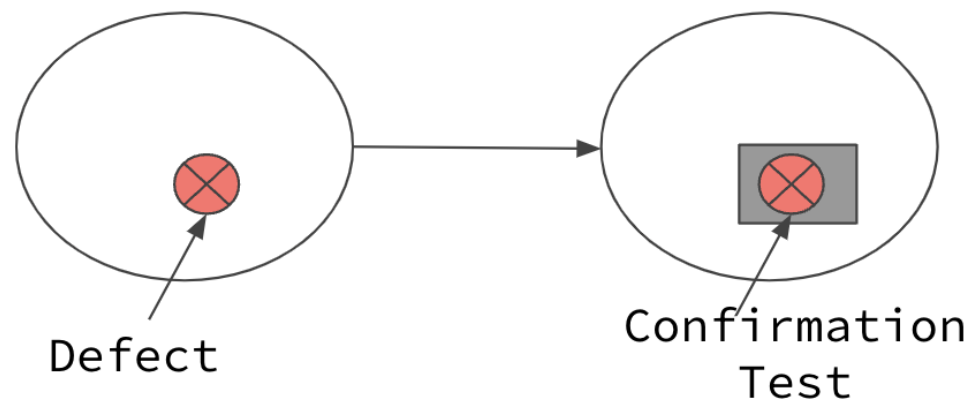
- Kiểm thử hộp trắng (White-box testing)

Kỹ thuật hộp trắng tiêu biểu	Mục tiêu
Statement Coverage	Đảm bảo tất cả dòng mã được thực thi
Branch Coverage	Bao phủ các nhánh điều kiện
Path Testing	Bao phủ các đường đi logic
Condition Coverage	Đảm bảo các điều kiện trong biểu thức được đánh giá đủ true/false
Multiple Condition Testing	Kiểm tra tất cả tổ hợp điều kiện logic

- 2.1 Kiểm thử trong Ngữ cảnh của Chu kỳ Phát triển Phần mềm
- 2.2 Các mức độ kiểm thử & Các loại kiểm thử
- **2.3 Kiểm thử hồi quy**

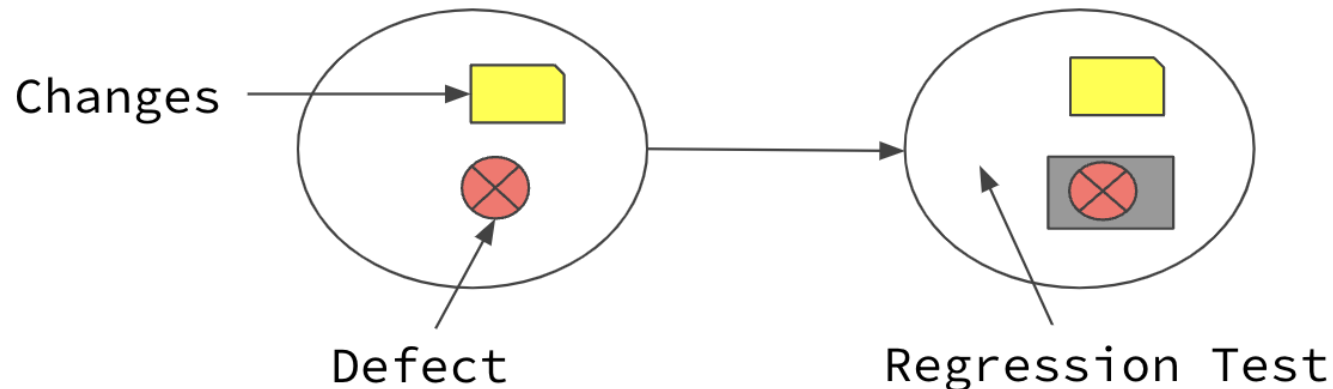
2.3 Kiểm thử hồi quy (Regression Testing)

- **Kiểm thử xác nhận (Confirmation testing)**
 - Xác nhận rằng một lỗi ban đầu (defect) đã được khắc phục thành công. Tùy thuộc vào mức độ rủi ro, người ta có thể kiểm thử phiên bản đã sửa của phần mềm theo các cách, bao gồm:
 - Thực hiện tất cả các tests trước đây đã bị failure do defect.
 - Hoặc thêm các tests mới để bao phủ bất kỳ thay đổi nào cần thiết để khắc phục lỗi.



2.3 Kiểm thử hồi quy (Regression Testing)

- **Kiểm thử hồi quy (Regression testing)**
 - Xác nhận rằng không có hậu quả tiêu cực nào xảy ra do một sự thay đổi, thay đổi có thể là:
 - Sửa lỗi đã được kiểm thử xác nhận
 - Thay đổi liên đến đến môi trường



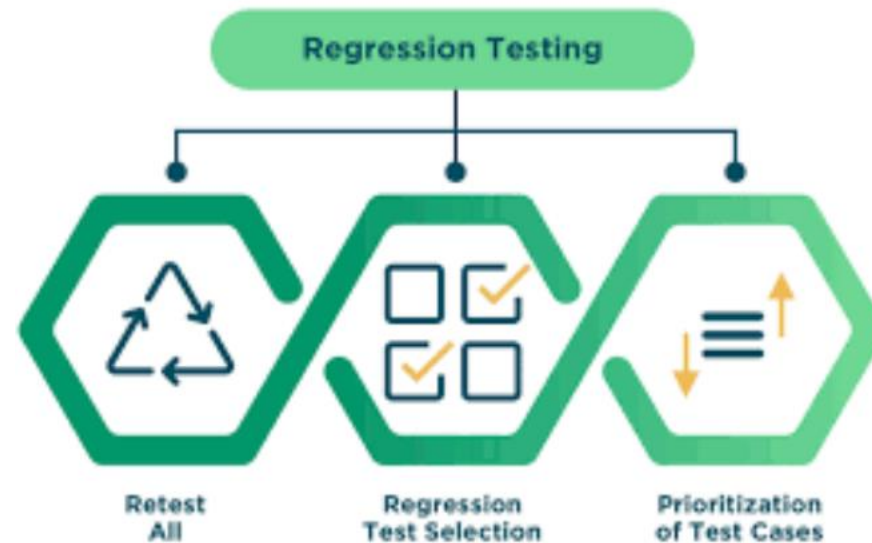
So sánh: Regression và Confirmation



Nội dung	Confirmation Test	Regression Test
Kiểm tra	Lỗi đã fix có hết chưa	Chức năng cũ có bị ảnh hưởng không
Phạm vi	Chỗ lỗi cụ thể	Rộng hơn – xung quanh chỗ thay đổi
Khi nào	Ngay sau khi fix bug	Sau khi merge/sửa code

2.3 Kiểm thử hồi quy (Regression Testing)

- Kiểm thử hồi quy (Regression testing)
- Nên thực hiện phân tích ảnh hưởng trước để tối ưu hóa phạm vi của kiểm thử hồi quy.
- Bộ kiểm thử hồi quy là một ứng cử viên cho việc tự động hóa.



2.4 Quiz

